# The OpenMP

# OpenMP: Work-Sharing Constructs

**Work-sharing constructs** are used to **divide work among multiple threads** in a team. They do **not create new threads**; instead, they distribute tasks.

- The "for" Work-Sharing construct splits up loop iterations among the threads in a team

```
#pragma omp parallel
#pragma omp for
        for (I=0;I<N;I++){
                NEAT_STUFF(I);
        }
```

**By default, there is a barrier at the end of the "omp for". Use the "nowait" clause to turn off the barrier.**

***#pragma omp for nowait***

**"nowait" is useful between two consecutive, independent omp for loops.**

# OpenMP: Work-Sharing Constructs

## A motivating example

```
#pragma omp parallel
{
    #pragma omp for
    for(int i=0; i<1000; i++)
    {
        process(i);
    }
}
```

If there are 4 threads:

| Thread | Iterations |
|---|---|
| T0 | 0-249 |
| T1 | 250-499 |
| T2 | 500-749 |
| T3 | 750-999 |

# OpenMP: Work-Sharing Constructs

## sections Construct

Used when different tasks are independent and can run simultaneously.

```
#pragma omp parallel
{
   #pragma omp sections
   {
      #pragma omp section
      {
         printf("Read data\n");
      }

      #pragma omp section
      {
         printf("Process data\n");
      }

      #pragma omp section
      {
         printf("Write data\n");
      }
   }
}
```

divides different independent blocks of code among different threads.

| Thread | Task |
|---|---|
| T0 | Read data |
| T1 | Process data |
| T2 | Write data |

# OpenMP: Work-Sharing Constructs

## single Construct

Only **one thread** executes a block of code.

```
#pragma omp parallel
{
    #pragma omp single
    {
        printf("Executed once\n");
    }
}
```

# OpenMP: Work-Sharing Constructs

- The master construct denotes a structured block that is only executed by the master thread. The other threads just skip it (no synchronization is implied).

```
#pragma omp parallel private (tmp)
{
        do_many_things();
#pragma omp master
        {    exchange_boundaries();   }
#pragma barrier
        do_many_other_things();
}
```

master thread is thread 0

# OpenMP: Work-Sharing Constructs

- The single construct denotes a block of code that is executed by only one thread.
- A barrier is implied at the end of the single block.

```
#pragma omp parallel private (tmp)
{
        do_many_things();
#pragma omp single
        {    exchange_boundaries();   }
        do_many_other_things();
}
```

# OpenMP: Work-Sharing Constructs

## Implicit Barrier

At the end of most work-sharing constructs, OpenMP inserts a **barrier** automatically.

```
#pragma omp for
for(int i=0; i<100; i++)
{
    work(i);
}

// all threads wait here
```

## Removing the Barrier with nowait

```
#pragma omp for nowait
for(int i=0; i<100; i++)
{
    work(i);
}
```

**When to Use:** Use nowait when the next work is independent.

# OpenMP: Work-Sharing Constructs

## Combined parallel/work-share

- OpenMP* shortcut: Put the "parallel" and the work-share on the same line

```
double res[MAX]; int i;
#pragma omp parallel
{
  #pragma omp for
  for (i=0;i< MAX; i++) {
      res[i] = huge();
  }
}
```

```
double res[MAX]; int i;
#pragma omp parallel for
  for (i=0;i< MAX; i++) {
      res[i] = huge();
  }
```

These are equivalent

- **There's also a "parallel sections" construct.**

# Data Environment: Default storage attributes

Data Environment: How variables are stored and accessed by different threads during parallel execution.

- **Shared Memory programming model:**
  - **Most variables are shared by default**
- **Global variables are SHARED among threads**
  - **C: File scope variables, static**
- **But not everything is shared...**
  - **Stack variables in sub-programs called from parallel regions are PRIVATE**
  - **Automatic variables within a statement block are PRIVATE.**

# Data Environment: Default storage attributes

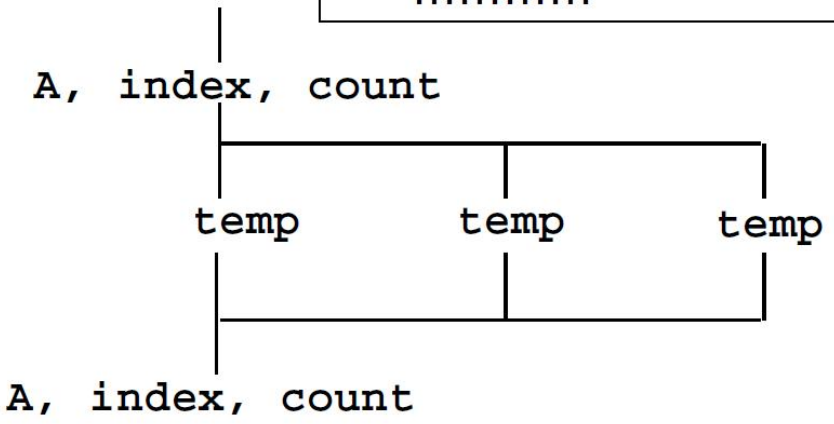
## Data Sharing Examples

```
    program sort
    common /input/ A(10)
    integer index(10)
!$OMP PARALLEL
    call work(index)
!$OMP END PARALLEL
    print*, index(1)
```

```
subroutine work (index)
common /input/ A(10)
integer index(*)
real temp(10)
integer count
save count
     ……
```

**A, index and count are shared by all threads.**

**temp is local to each thread**



# Data Environment: Changing storage attributes

- One can selectively change storage attributes constructs using the following clauses*
  - SHARED
  - PRIVATE
  - FIRSTPRIVATE
  - THREADPRIVATE

**All the clauses on this page apply to the *OpenMP Construct* NOT the entire region.**

- The value of a private inside a parallel loop can be transmitted to a global value outside the loop with:
  - LASTPRIVATE

private during the parallel work + copy the value from the last iteration back to the original variable.

- The default status can be modified with:
  - DEFAULT (PRIVATE | SHARED | NONE)

**All data clauses apply to parallel regions and work sharing constructs except "shared" which only applies to parallel regions.**

# Data Environment: Changing storage attributes

## Private Clause

- private(var) creates a local copy of var for each thread.
  - The value is uninitialized
  - Private copy is *not* storage-associated with the original
  - The original is undefined at the end

```
      program wrong
      IS = 0
C$OMP PARALLEL DO PRIVATE(IS)
      DO J=1,1000
           IS = IS + J
      END DO
      print *, IS
```

IS was not initialized

Regardless of initialization, IS is undefined at this point

# Firstprivate Clause

firstprivate is similar to private, but each thread's private copy is initialized with the original variable's value before the parallel work starts.

- Firstprivate is a special case of private.
  - Initializes each private copy with the corresponding value from the master thread.

```
      program almost_right
      IS = 0
C$OMP PARALLEL DO FIRSTPRIVATE(IS)
      DO J=1,1000
           IS = IS + J
1000  CONTINUE
      print *, IS
```

Each thread gets its own IS with an initial value of 0

Regardless of initialization, IS is undefined at this point

# Data Environment: Changing storage attributes

## Lastprivate Clause

- Lastprivate passes the value of a private from the last iteration to a global variable.

```
      program closer
      IS = 0
C$OMP PARALLEL DO FIRSTPRIVATE(IS)
C$OMP+ LASTPRIVATE(IS)
      DO J=1,1000
           IS = IS + J
1000  CONTINUE
      print *, IS
```

Each thread gets its own IS with an initial value of 0

IS is defined as its value at the "last sequential" iteration (I.e. for J=1000)

# OpenMP: A Data Environment Test

- Consider this example of PRIVATE and FIRSTPRIVATE

```
      variables A,B, and C = 1
C$OMP PARALLEL PRIVATE(B)
C$OMP& FIRSTPRIVATE(C)
```

- Are A,B,C local to each thread or shared inside the parallel region?
- What are their initial values inside and after the parallel region?

**Inside this parallel region ...**

- **"A" is shared by all threads; equals 1**
- **"B" and "C" are local to each thread.**
  - **B's initial value is undefined**
  - **C's initial value equals 1**

**Outside this parallel region ...**

- **The values of "B" and "C" are undefined.**

# OpenMP: Reduction

Combines an accumulation operation across threads:

reduction (op : list)

if addition then initialize with 0 and if operation is multiplication then iniatialize with 1

Inside a parallel or a work-sharing construct:

- A local copy of each list variable is made and initialized depending on the “op” (e.g. 0 for “+”).
- Compiler finds standard reduction expressions containing “op” and uses them to update the local copy.
- Local copies are reduced into a single value and combined with the original global value.

The variables in “list” must be shared in the enclosing parallel region.

# OpenMP: Reduction

## Example

- Remember the code we used to demo private, firstprivate and lastprivate.

```
      program closer
      IS = 0
      DO J=1,1000
           IS = IS + J
1000  CONTINUE
      print *, IS
```

- **Here is the correct way to parallelize this code.**

```
      program closer
      IS = 0
#pragma omp parallel for reduction(+:IS)
      DO J=1,1000
           IS = IS + J
1000  CONTINUE
      print *, IS
```

# OpenMP: Reduction operands/initial-values

- A range of associative operands can be used with reduction:
- Initial values are the ones that make sense mathematically.

| Operand | Initial value |
|---|---|
| + | 0 |
| * | 1 |
| - | 0 |
| .AND. | .true. |
| .OR. | .false. |
| .neqv. | .false. |
| .eqv. | .true. |
| MIN* | Largest pos. number |
| MAX* | Most neg. number |

| Operand | Initial value |
|---|---|
| iand | All bits on |
| ior | 0 |
| ieor | o |
| & | ~0 |
| \| | 0 |
| ^ | 0 |
| && | 1 |
| \|\| | 0 |

# Default Clause

- Note that the default storage attribute is DEFAULT(SHARED) (so no need to use it)
- To change default: DEFAULT(PRIVATE)
  - *each* variable in *static* extent of the parallel region is made private as if specified in a private clause
  - mostly saves typing
- DEFAULT(NONE): *no* default for variables in static extent. Must list storage attribute for each variable in static extent

must define as private or shared in child program

**C/C++ only has default(shared) or default(none).**

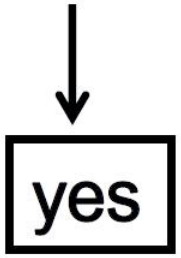
# Default Clause

## Example

```
      itotal = 1000
C$OMP PARALLEL PRIVATE(np, each)
      np = omp_get_num_threads()
      each = itotal/np
      .....
C$OMP END PARALLEL
```

```
      itotal = 1000
C$OMP PARALLEL DEFAULT(PRIVATE) SHARED(itotal)
      np = omp_get_num_threads()
      each = itotal/np
      .....
C$OMP END PARALLEL
```

**Are these two codes equivalent?**



# Threadprivate

- Makes global data private to a thread
  - C: File scope and static variables
- Different from making them PRIVATE
  - with PRIVATE global variables are masked.
  - THREADPRIVATE preserves global scope within each thread
- Threadprivate variables can be initialized using COPYIN or by using DATA statements.

# Threadprivate

## Example

**Consider two different routines called within a parallel region.**

```
      subroutine poo
      parameter (N=1000)
      common/buf/A(N),B(N)
!$OMP THREADPRIVATE(/buf/)
      do i=1, N
        B(i)= const* A(i)
      end do
      return
      end
```

```
      subroutine bar
      parameter (N=1000)
      common/buf/A(N),B(N)
!$OMP THREADPRIVATE(/buf/)
      do i=1, N
        A(i) = sqrt(B(i))
      end do
      return
      end
```

**Because of the threadprivate construct, each thread executing these routines has its own copy of the common block /buf/.**

# Copyin

**You initialize threadprivate data using a copyin clause.** copyin = Copy the master's value IN to all threads.

```
      parameter (N=1000)
      common/buf/A(N)
!$OMP THREADPRIVATE(/buf/)

C Initialize the A array
      call init_data(N,A)

!$OMP PARALLEL COPYIN(A)

 … Now each thread sees threadprivate array A initialied
 … to the global value set in the subroutine init_data()

!$OMP END PARALLEL

end
```

# Copyprivate

Used with a single region to broadcast values of privates from one member of a team to the rest of the team.

```
#include <omp.h>
void input_parameters (int, int); // fetch values of input parameters
void do_work(int, int);

void main()
{
  int Nsize, choice;

  #pragma omp parallel private (Nsize, choice)
  {
     #pragma omp single copyprivate (Nsize, choice)
          input_parameters (Nsize, choice);

     do_work(Nsize, choice);
  }
}
```

copyprivate = one thread's private value is shared with the others

# OpenMP: Synchronization

When multiple threads execute concurrently, they often need to **coordinate** with each other. This coordination is called **synchronization**.

Without synchronization, threads may:

- ❑ Access shared data simultaneously.
- ❑ Produce incorrect results.
- ❑ Create race conditions.
- ❑ Read incomplete data.

**OpenMP provides:**

- ❑ Barrier
- ❑ Critical
- ❑ Atomic
- ❑ Ordered
- ❑ Master
- ❑ Single
- ❑ Flush
- ❑ Locks

# OpenMP: Synchronization

## Why Synchronization is Needed?

Consider the following code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int counter = 0;

    #pragma omp parallel
    {
        counter++;
    }

    printf("Counter = %d\n", counter);
}
```

Suppose 4 threads execute:

| Thread | Action |
|---|---|
| T0 | Read counter = 0 |
| T1 | Read counter = 0 |
| T2 | Read counter = 0 |
| T3 | Read counter = 0 |
| All | Add 1 |
| All | Write back 1 |

Expected result: Counter = 4

Possible result: Counter = 1

This problem is called a **race condition**.

Synchronization prevents such problems.

# OpenMP: Synchronization

## Barrier Directive

A barrier forces all threads to wait until every thread reaches that point.

```
#pragma omp parallel
{
    printf("Thread %d working\n",
            omp_get_thread_num());

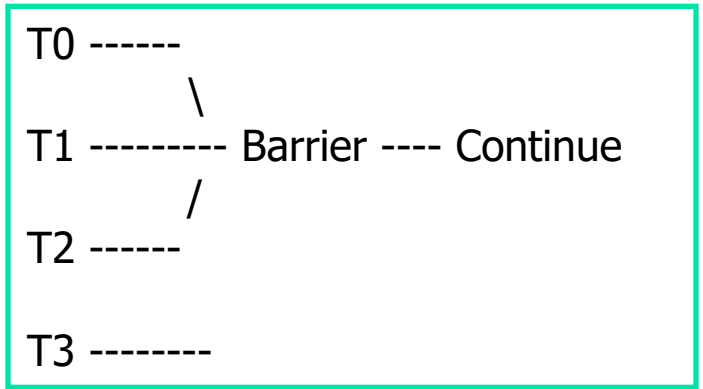
    #pragma omp barrier

    printf("Thread %d passed barrier\n",
            omp_get_thread_num());
}
```

```
T0 working
T1 working
T2 working
T3 working

(All wait)

T0 passed barrier
T1 passed barrier
T2 passed barrier
T3 passed barrier
```




All threads must arrive before any continue.

# OpenMP: Synchronization

**Atomic Directive**

Used for a single memory update operation.

```
int counter = 0;

#pragma omp parallel
{
    #pragma omp atomic
    counter++;
}
```

**Result:** Counter = 4

# OpenMP: Synchronization

## Ordered Directive

Ensures loop iterations execute in a specific order.

```
#pragma omp parallel for ordered
for(int i=0;i<5;i++)
{
    // Parallel work

    #pragma omp ordered
    {
        printf("%d\n", i);
    }
}
```

Output:

0
1
2
3
4

Without Ordered

2
4
1
0
3

# OpenMP: Synchronization

**Master Directive**

Only the master thread (Thread 0) executes the block.

```
#pragma omp parallel
{
    #pragma omp master
    {
        printf("Master thread only\n");
    }
}
```

**Output:** Master thread only

- No barrier at the end.
- Other threads continue immediately.

# OpenMP: Synchronization

**Single Directive**

One arbitrary thread executes the block.

```
#pragma omp parallel
{
    #pragma omp single
    {
        printf("Executed once\n");
    }
}
```

Output: Executed once

| Feature | Master | Single |
|---|---|---|
| Executed by | Thread 0 | Any thread |
| Barrier at end | No | Yes (default) |
| Used for | Control tasks | One-time tasks |

# OpenMP: Synchronization

## Flush Directive

Makes memory updates visible to other threads.

```
int flag = 0;

#pragma omp parallel sections
{
   #pragma omp section
   {
      flag = 1;
      #pragma omp flush(flag)
   }

   #pragma omp section
   {
      while(flag == 0)
      {
         #pragma omp flush(flag)
      }

      printf("Flag detected\n");
   }
}
```

The OpenMP flush directive is used to establish memory consistency/visibility between threads, ensuring that updates to specified variables become visible to other threads according to the OpenMP memory model.

**Purpose:**

Ensures threads see the latest value from memory.

# OpenMP: Synchronization

## OpenMP Locks

Locks provide manual synchronization.

```
omp_lock_t lock;

omp_init_lock(&lock);

#pragma omp parallel
{
    omp_set_lock(&lock);

    printf("Thread %d inside\n",
           omp_get_thread_num());

    omp_unset_lock(&lock);
}

omp_destroy_lock(&lock);
```

Initialize Lock

Set Lock

Unlock

Destroy Lock

A lock allows only one thread at a time to enter a protected section of code.

# OpenMP: Synchronization

| Construct | Purpose |
|---|---|
| Barrier | Wait for all threads |
| Critical | One thread at a time for a block |
| Atomic | One thread at a time for a memory update |
| Ordered | Preserve loop order |
| Master | Execute by thread 0 only |
| Single | Execute by one thread only |
| Flush | Update shared memory visibility |
| Lock | Manual synchronization |
| Nowait | Remove implicit barrier |